



Data Structures

CPT102 – Lecture 13
Erick Purwanto



Xi'an Jiaotong-Liverpool University

西交利物浦大學

CPT102 Data Structures

Lecture 13

Hashing

Welcome!

- Welcome to Lecture 13 !
- In this lecture we are going to
 - learn about hashing
 - use it to create hash table data structure
 - learn to solve issues related to hashing
 - solve collision handling issue with separate chaining
 - solve hash code distribution issue with resizing hash tables and good hash functions

Hash Tables



- We will continue our goal in developing data structures that support fast insert and search
- In this lecture, we will build a hash-based Set
 - and introduce hashing techniques
 - Set, like Map, support inserting keys and searching for keys
 - inserting keys `add(x)`
 - searching keys `contains(x)` return true or false
 - similar ideas apply for Map, just attach a value to the key

Last Lecture

- In **Lecture 12**, we have shown:



Data Structures	get(x)	put(x, val)	Notes
Ordered Arrays	$O(\log N)$	$O(N)$	Binary search, but need to shift
BSTs	$O(N)$	$O(N)$	Reduced to a chain in the worst case
2-3 Trees	$O(\log N)$	$O(\log N)$	Perfect balanced, but not a binary tree
Red-Black Trees	$O(\log N)$	$O(\log N)$	Perfect black balanced

In-Class Quiz 1

- Do you *agree* or *disagree* with this statement:

"Using BSTs or Red-Black Trees, we can store and search people's data with **their face images** as keys"

- ***Explain*** why you agree or disagree!

Hashing

- In this lecture, we will develop hash tables that can store *any* keys
 - and can do searching in (average/amortized) constant time
- Let us start simple by *only* storing *integer* data
 - and call our set implementation **INTSet**
 - remember, our goal is to *make inserting and searching as fast as possible!*
 - inserting keys `add(x)`
 - searching keys `contains(x)` return true or false

INTSet

- Idea: Use an array of boolean that is indexed by the integer data
 - that is, the index is actually the data
 - initially all boolean values are false

```
INTSet set;  
set = new INTSet();
```

initial empty set

F	0
F	1
F	2
F	3
F	4
F	5
F	6
F	7
F	8
F	9
F	10
F	11
F	12
F	13
F	14
F	15

...

INTSet

- Idea: Use an array of boolean that is indexed by the integer data
 - that is, the index is actually the data
 - initially all boolean values are false
 - when an item is added, set corresponding index to true

```
INTSet set;  
set = new INTSet();  
set.add(5);
```

set contains 5



F	0
F	1
F	2
F	3
F	4
T	5
F	6
F	7
F	8
F	9
F	10
F	11
F	12
F	13
F	14
F	15

...

INTSet

- Idea: Use an array of boolean that is indexed by the integer data
 - that is, the index is actually the data
 - initially all boolean values are false
 - when an item is added, set corresponding index to true

```
INTSet set;  
set = new INTSet();  
set.add(5);  
set.add(2);
```

set contains 5, 2



F	0
F	1
T	2
F	3
F	4
T	5
F	6
F	7
F	8
F	9
F	10
F	11
F	12
F	13
F	14
F	15

...

INTSet

- Idea: Use an array of boolean that is indexed by the integer data
 - that is, the index is actually the data
 - initially all boolean values are false
 - when an item is added, set corresponding index to true

```
INTSet set;  
set = new INTSet();  
set.add(5);  
set.add(2);  
set.add(10);
```

contains(i) simply check
whether value at index i is T or F

set contains 5, 2, 10

F	0
F	1
T	2
F	3
F	4
T	5
F	6
F	7
F	8
F	9
T	10
F	11
F	12
F	13
F	14
F	15

...

INTSet Implementation

- The implementation is very simple:

```
public class INTSet {  
    private boolean[] items;  
  
    public INTSet() {  
        items = new boolean[2000000000];  
    }  
  
    public void add(int i) {  
        items[i] = true;  
    }  
  
    public boolean contains(int i) {  
        return items[i];  
    }  
}
```

roughly the number of integers

since accessing elements in
an array is constant time,
these operations = $O(1)$ time

In-Class Quiz 2

- What are the **potential issues** to this approach?

```
public class INTSet {  
    private boolean[] items;  
  
    public INTSet() {  
        items = new boolean[2000000000];  
    }  
  
    public void add(int i) {  
        items[i] = true;  
    }  
  
    public boolean contains(int i) {  
        return items[i];  
    }  
}
```

INTSet Drawbacks

- There are some obvious problems with this implementation:

```
public class INTSet {  
    private boolean[] items;  
  
    public INTSet() {  
        items = new boolean[2000000000];  
    }  
  
    public void add(int i) {  
        items[i] = true;  
    }  
  
    public boolean contains(int i) {  
        return items[i];  
    }  
}
```

1. extremely wasteful of memory!
even if you only want to store
a few integers you still need
two billion booleans

2. you can only store integers
need a way to generalize

essentially, when we can solve
those two issues, we will get
ourselves a hash table !

ENGSet

- Now suppose we want to use the same idea, but we want to store **lowercase English words** instead of just integers
- We want a set ENGSet that can:

```
ENGSet set;  
set = new ENGSet();  
set.add("cat");  
set.add("dog");  
set.add("animal");
```

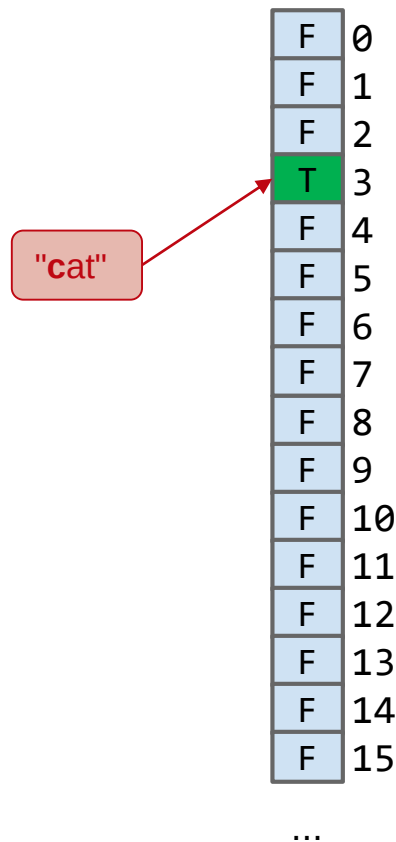
but where should we put
"cat", "dog", and "animal" ?

F	0
F	1
F	2
F	3
F	4
F	5
F	6
F	7
F	8
F	9
F	10
F	11
F	12
F	13
F	14
F	15

...

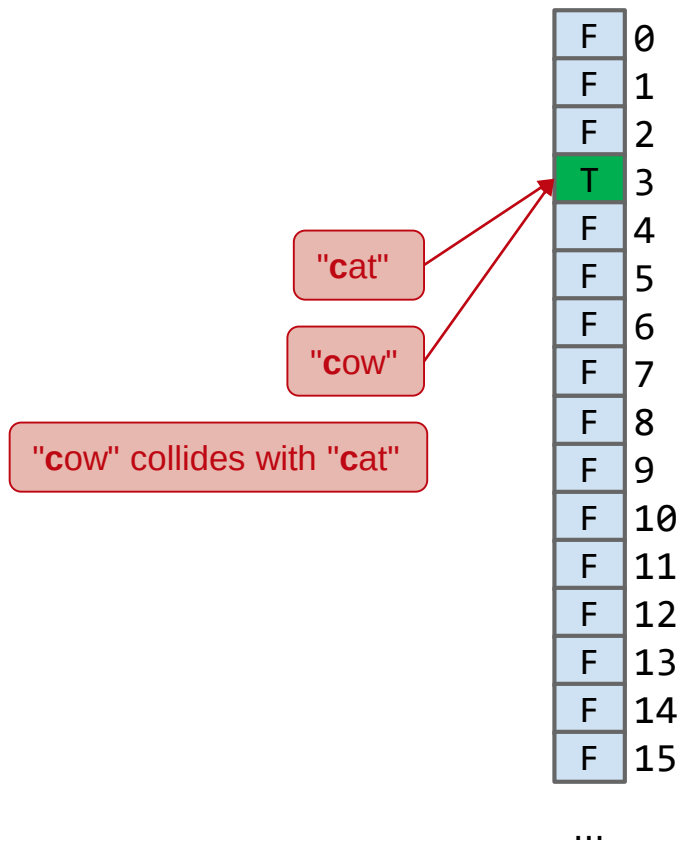
Add into ENGSet (1)

- Suppose we want to add("cat")
 - what is the index for "cat" ?
 - idea: use the first letter as index
a = 1, b = 2, **c = 3**, ..., z = 26



Add into ENGSet (2)

- Suppose we want to add("cat")
 - what is the index for "cat" ?
 - idea: use the first letter as index
a = 1, b = 2, **c = 3**, ..., z = 26
 - however, contains("cow") → T
 - we have a **collision** !



Add into ENGSet (3)

- Suppose we want to add("cat") with no collision
 - to avoid collision, idea: use **all** letters by multiplying each by a power of 27
 - 27 because there are 26 letters in English alphabet, and multiply by powers like a decimal number in base 10, e.g.
 $521_{10} = (5 \times 10^2) + (2 \times 10^1) + (1 \times 10^0)$

0	F	
1	F	a
2	F	b
3	F	c
4	F	d

...

25	F	y
26	F	z

...

2233	F	cas
2234	F	cat
2235	F	cau

...

Add into ENGSet (4)

- Suppose we want to add("cat") with no collision
 - to avoid collision, idea: use **all** letters by multiplying each by a power of 27
 - 27 because there are 26 letters in English alphabet, and multiply by powers like a decimal number in base 10, e.g.
 $521_{10} = (5 \times 10^2) + (2 \times 10^1) + (1 \times 10^0)$
 - so, the index of "cat" is $(3 \times 27^2) + (1 \times 27^1) + (20 \times 27^0) = 2234$

since we pick a base ≥ 26 , it is guaranteed to give each lowercase English word a unique number!
we will have **no** collision!

0	F	
1	F	a
2	F	b
3	F	c
4	F	d
...		
25	F	y
26	F	z
...		
2233	F	cas
2234	T	cat
2235	F	cau
...		

"cat"

Convert English String to Integer

- Method `letterNum` converts letter to number, and `englishToInt` computes the index

```
/* Converts ith character of String to a letter number.
 * e.g. 'a' -> 1, 'b' -> 2, 'z' -> 26 */
private static int letterNum(String s, int i) {
    int ithChar = s.charAt(i);
    if ((ithChar < 'a') || (ithChar > 'z'))
        { throw new IllegalArgumentException(); }
    return ithChar - 'a' + 1;
}

private static int englishToInt(String s) {
    int intRep = 0;
    for (int i = 0; i < s.length(); i++) {
        intRep = intRep * 27;
        intRep = intRep + letterNum(s, i);
    }
    return intRep;
}
```

in Java, char 'a' is equivalent to 97
this will give 1 to 'a', 2 to 'b', ...

ENGSet Implementation

- Similar to INTSet, simply needs to compute the index first

```
public class ENGSet {  
    private boolean[] items;  
  
    public ENGSet() {  
        items = new boolean[2000000000];  
    }  
  
    public void add(String s) {  
        items[englishToInt(s)] = true;  
    }  
  
    public boolean contains(String s) {  
        return items[englishToInt(s)];  
    }  
}
```

compute index of s

0	F	
1	F	a
2	F	b
3	F	c
4	F	d

...

25	F	y
26	F	z

...

2233	F	cas
2234	F	cat
2235	F	cau

...

ASCII Characters

- In addition to lowercase letter, what if we want to store uppercase? numbers? special chars?
- We can use **ASCII encoding value** as the index
 - each char is assigned a value between 0 and 127: `char c = 'a'` is equiv with `char c = 97` in Java

characters 33 - 126
are printable



the others do other
things, like the
new line character

33	!
34	"
35	#
36	\$
37	%
38	&
39	'
40	(
41	)
42	*
43	+
44	,
45	-
46	.
47	/
48	0

49	1
50	2
51	3
52	4
53	5
54	6
55	7
56	8
57	9
58	:
59	;
60	<
61	=
62	>
63	?
64	@

65	A
66	B
67	C
68	D
69	E
70	F
71	G
72	H
73	I
74	J
75	K
76	L
77	M
78	N
79	O
80	P

81	Q
82	R
83	S
84	T
85	U
86	V
87	W
88	X
89	Y
90	Z
91	[
92	\
93	]
94	^
95	_
96	`

97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	i
106	j
107	k
108	l
109	m
110	n
111	o
112	p

113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z
123	{
124	
125	}
126	~

ASCII-based SET

- Maximum possible value for english-only text including punctuation is 126
- We can use 126 as our base in order to ensure unique values for all possible strings
- For examples:
 - $\text{bee}_{126} = (98 \times 126^2) + (101 \times 126^1) + (101 \times 126^0) = 1,568,675$
 - $\text{2pac}_{126} = (50 \times 126^3) + (112 \times 126^2) + (97 \times 126^1) + (99 \times 126^0) = 101,809,233$

ASCII-based SET Implementation

- We can use the following method to convert the index:

```
public static int asciiToInt(String s) {  
    int intRep = 0;  
    for (int i = 0; i < s.length(); i++) {  
        intRep = intRep * 126;  
        intRep = intRep + s.charAt(i);  
    }  
    return intRep;  
}
```

but what we want store even more characters? e.g., Chinese characters?

Unicode Characters



- We can also use Unicode encoding value, also supported in Java
 - for example, `char c = '鳌'` is equivalent to `char c = 40140`

U+2F00	U+2F01	U+2F02	U+2F03	U+2F04	U+2F05	U+2F06	U+2F07	U+2F08	U+2F09	U+2F0A	U+2F0B	U+2F0C	U+2F0D	U+2F0E	U+2F0F
一	丨	丶	ノ	乙	丅	二	十	人	儿	入	八	冂	冃	冂	几
U+2F10	U+2F11	U+2F12	U+2F13	U+2F14	U+2F15	U+2F16	U+2F17	U+2F18	U+2F19	U+2F1A	U+2F1B	U+2F1C	U+2F1D	U+2F1E	U+2F1F
凵	刀	力	勹	匕	匚	匚	十	卜	卩	厂	厶	又	口	口	土
U+2F20	U+2F21	U+2F22	U+2F23	U+2F24	U+2F25	U+2F26	U+2F27	U+2F28	U+2F29	U+2F2A	U+2F2B	U+2F2C	U+2F2D	U+2F2E	U+2F2F
士	夕	夕	夕	大	女	子	冂	寸	小	尢	尸	巾	山	叕	工
U+2F30	U+2F31	U+2F32	U+2F33	U+2F34	U+2F35	U+2F36	U+2F37	U+2F38	U+2F39	U+2F3A	U+2F3B	U+2F3C	U+2F3D	U+2F3E	U+2F3F
己	巾	干	幺	广	廾	弋	弓	厶	彡	彳	心	戈	户	手	
U+2F40	U+2F41	U+2F42	U+2F43	U+2F44	U+2F45	U+2F46	U+2F47	U+2F48	U+2F49	U+2F4A	U+2F4B	U+2F4C	U+2F4D	U+2F4E	U+2F4F
支	支	文	斗	斤	方	无	日	日	月	木	欠	止	歹	殳	母
U+2F50	U+2F51	U+2F52	U+2F53	U+2F54	U+2F55	U+2F56	U+2F57	U+2F58	U+2F59	U+2F5A	U+2F5B	U+2F5C	U+2F5D	U+2F5E	U+2F5F
比	毛	氏	气	水	火	爪	父	爻	爿	片	牙	牛	犬	玄	玉
U+2F60	U+2F61	U+2F62	U+2F63	U+2F64	U+2F65	U+2F66	U+2F67	U+2F68	U+2F69	U+2F6A	U+2F6B	U+2F6C	U+2F6D	U+2F6E	U+2F6F
瓜	瓦	甘	生	用	田	疋	广	𠂔	白	皮	皿	目	矛	矢	石
U+2F70	U+2F71	U+2F72	U+2F73	U+2F74	U+2F75	U+2F76	U+2F77	U+2F78	U+2F79	U+2F7A	U+2F7B	U+2F7C	U+2F7D	U+2F7E	U+2F7F
示	丙	禾	穴	立	竹	米	糸	缶	网	羊	羽	老	而	耒	耳
U+2F80	U+2F81	U+2F82	U+2F83	U+2F84	U+2F85	U+2F86	U+2F87	U+2F88	U+2F89	U+2F8A	U+2F8B	U+2F8C	U+2F8D	U+2F8E	U+2F8F
聿	肉	臣	自	至	白	舌	舛	舟	艮	色	艸	虍	虫	血	行
U+2F90	U+2F91	U+2F92	U+2F93	U+2F94	U+2F95	U+2F96	U+2F97	U+2F98	U+2F99	U+2F9A	U+2F9B	U+2F9C	U+2F9D	U+2F9E	U+2F9F
衣	西	見	角	言	谷	豆	豕	豕	貝	赤	走	足	身	車	辛
U+2FA0	U+2FA1	U+2FA2	U+2FA3	U+2FA4	U+2FA5	U+2FA6	U+2FA7	U+2FA8	U+2FA9	U+2FAA	U+2FAB	U+2FAC	U+2FAD	U+2FAE	U+2FAF
辰	辵	邑	酉	采	里	金	長	門	阜	隹	雨	青	非	面	
U+2FB0	U+2FB1	U+2FB2	U+2FB3	U+2FB4	U+2FB5	U+2FB6	U+2FB7	U+2FB8	U+2FB9	U+2FBA	U+2FBB	U+2FBC	U+2FBD	U+2FBE	U+2FBF
革	韋	韭	音	頁	風	飛	食	首	香	馬	骨	高	髟	鬥	鬯
U+2FC0	U+2FC1	U+2FC2	U+2FC3	U+2FC4	U+2FC5	U+2FC6	U+2FC7	U+2FC8	U+2FC9	U+2FCA	U+2FCB	U+2FCC	U+2FCD	U+2FCE	U+2FCF
鬲	鬼	魚	鳥	鹵	鹿	麥	麻	黃	黍	黑	黽	鼈	鼎	鼓	鼠

just some tiny part,
the table is too big to display

Unicode-based SET

- The largest possible value for Chinese characters is 40,959
 - (probably more based on the most recent version of Unicode that is continuously updated)
- So we would need to use that number as our base if we want to have a *unique* representation for all possible strings of Chinese characters
- For example:

$$\begin{aligned}\text{守门员}_{40959} &= (23432 \times 40959^2) + (38376 \times 40959^1) + (21592 \times 40959^0) = \\ &= 39,312,024,869,368\end{aligned}$$



...		
39,312,024,869,367	F	守门员
39,312,024,869,368	T	守门员
39,312,024,869,369	F	守门员
...		

Integer Overflow

- However, it is not possible to have that large of an integer index in Java
- In Java, the largest possible integer is 2,147,483,647
 - If you go over this limit, you **overflow**, starting back over at the smallest integer, which is -2,147,483,648

```
int x = 2147483647;  
System.out.println(x);  
System.out.println(x + 1);
```

→ 2147483647

→ -2147483648

Overflow Causes Invalid Index and Collision

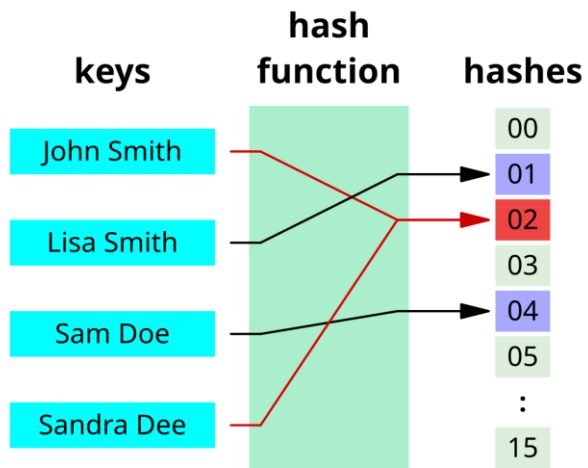
- Because Java has a maximum integer, we may not get the number we expect
 - With base 126, we will run into overflow *even* for short strings
For example: $\text{omens}_{126} = 28,196,917,171$
which is much greater than the maximum integer,
and `asciiToInt('omens')` will give us -1,867,853,901 instead
 - Overflow can result in collisions, causing incorrect answers

```
public void testOverflow() {  
    ENGSet set = new ENGSet();  
    set.add("melt banana");  
    set.contains("subterrestrial anticosmetic");  
    // asciiToInt for these strings is 839099497  
}
```

→ true

Hash Codes

- The term for the index/number we're computing is **hash codes / hashes**
- A hash code **projects** a value *from* a set with many (or even an infinite number of) members **to** a value from a set with a fixed number of (fewer) members
 - here, our target set is the set of Java integers, which is of size 4,294,967,296
 - thus, there can only be 4,294,967,296 possible hash codes



Pigeonhole Principle

- Pigeonhole principle tells us that if there are *more* than 4294967296 possible items, *multiple* items will share the *same* hash code
 - there are more than 4,294,967,296 **planets**
 - Each has mass, xPos, yPos, xVel, yVel, name
 - there are more than 4,294,967,296 **strings**
 - "one", "two", ..., "nineteen quadrillion", ...
- Thus, **collisions are inevitable** !



there are 9 boxes and 10 pigeons, **no matter** how you arrange the pigeons, it is guaranteed that **at least 1 box** is going to contain **more than 1 pigeon**

Two Fundamental Challenges

Two fundamental challenges:

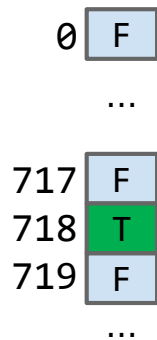
- How do we resolve the *hash code collisions*?
 - let us call this **collision handling**
- How do we compute a hash code for *arbitrary objects*?
 - let us call this **computing a hash code**



Handling Collision

- Suppose N items have the same hash code h

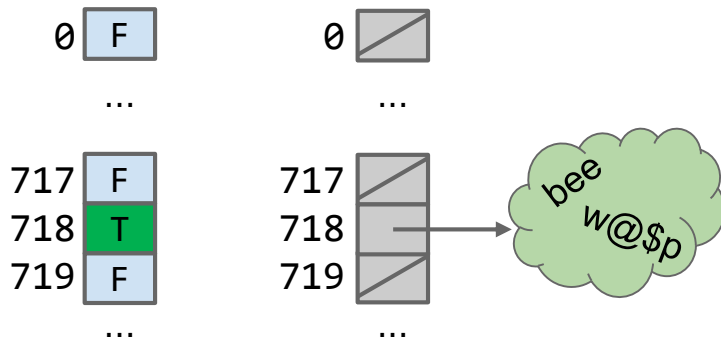
suppose hash code for both
"bee" and "w@\$p" is 718



Handling Collision

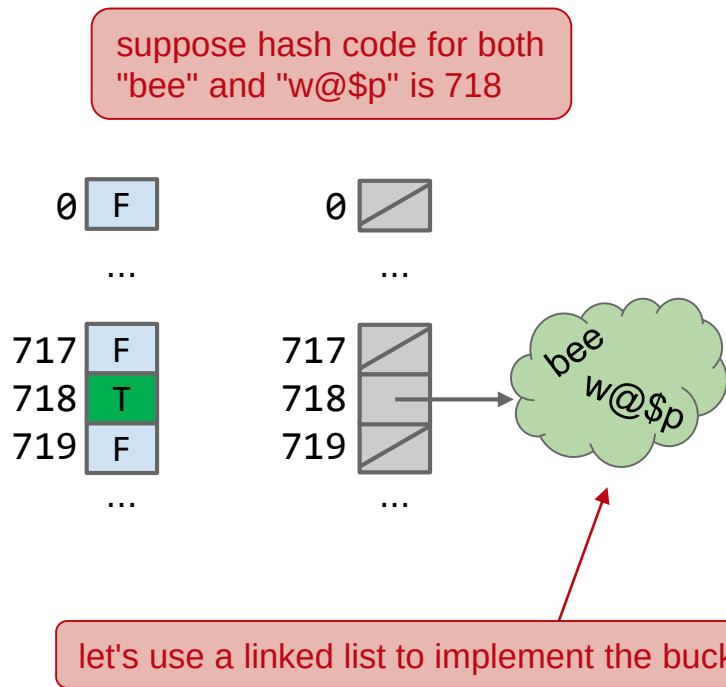
- Suppose N items have the same hash code h
 - instead of storing true in position h, store a **bucket** of these N items at position h

suppose hash code for both
"bee" and "w@\$p" is 718



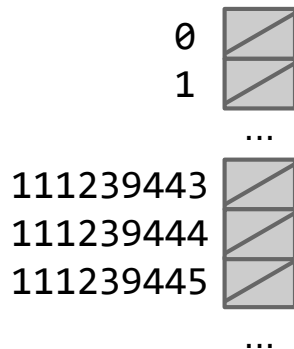
Handling Collision

- Suppose N items have the same hash code h
 - instead of storing true in position h, store a **bucket** of these N items at position h
- How to implement a bucket?
 - use a linked list
 - could use an array list
 - could also use an array set
 - any data structure which you can search and iterate
- This idea is called **separate chaining**



Separate Chaining

- In separate chaining, when an item x gets added at index h :
 - If bucket h is empty, we create a new list containing x and store it at index h
 - If bucket h is already a list, we add x to this list *if* it is *not* already present

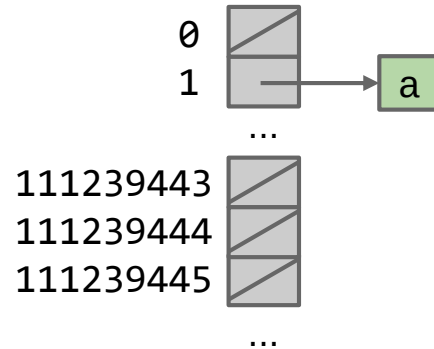


initially, all buckets
are empty

Separate Chaining

- In separate chaining, when an item x gets added at index h :
 - If bucket h is empty, we create a new list containing x and store it at index h
 - If bucket h is already a list, we add x to this list *if* it is *not* already present

`add("a")`

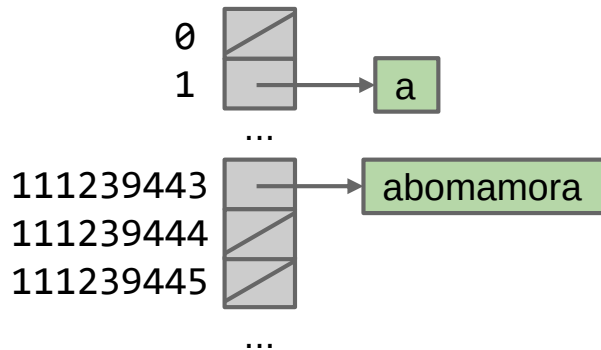


bucket 1 now has
a list of length 1

Separate Chaining

- In separate chaining, when an item x gets added at index h :
 - If bucket h is empty, we create a new list containing x and store it at index h
 - If bucket h is already a list, we add x to this list *if* it is *not* already present

```
add("a")  
add("abomamora")
```

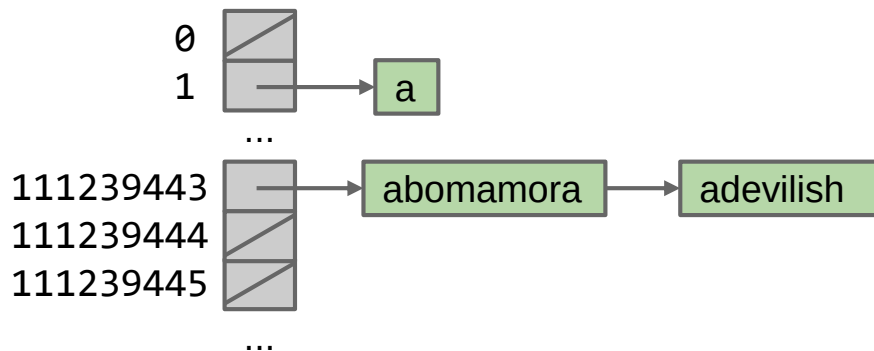


using `englishToInt`,
abomamora hashes to
111239443, create new list

Separate Chaining

- In separate chaining, when an item x gets added at index h :
 - If bucket h is empty, we create a new list containing x and store it at index h
 - If bucket h is already a list, we add x to this list *if* it is *not* already present

```
add("a")  
add("abomamora")  
add("adevilish")
```

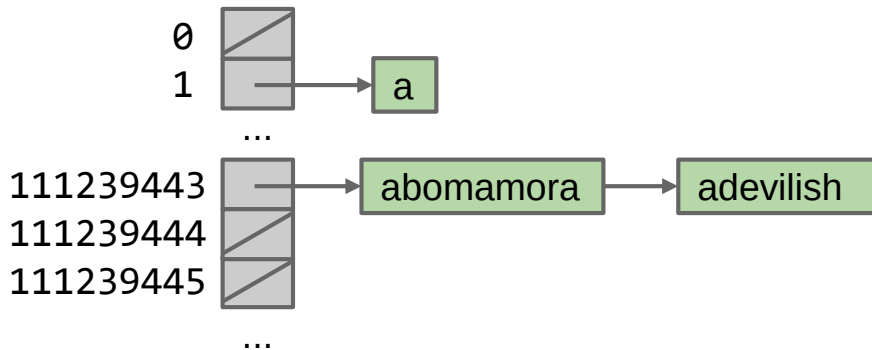


adevilish also hashes to
111239443, added in list

In-Class Quiz 3

- In separate chaining, when an item x gets added at index h :
 - If bucket h is empty, we create a new list containing x and store it at index h
 - If bucket h is already a list, we add x to this list *if* it is *not* already present

```
add("a")  
add("abomamora")  
add("adevilish")  
add("abomamora")
```

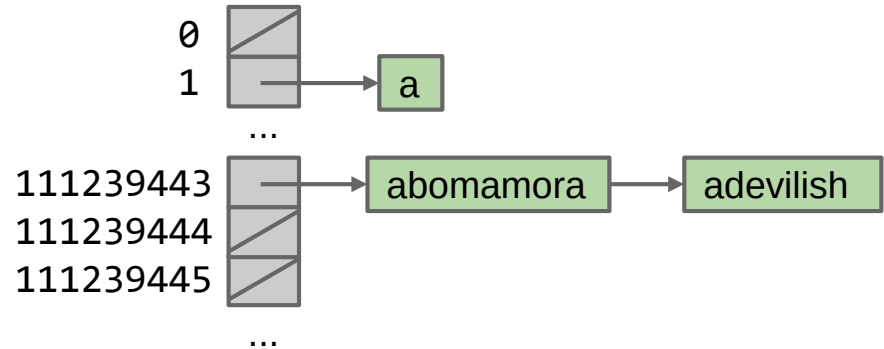


What will happen if we add **"abomamora"** ?

Separate Chaining

- In separate chaining, when an item x gets added at index h :
 - If bucket h is empty, we create a new list containing x and store it at index h
 - If bucket h is already a list, we add x to this list *if* it is *not* already present

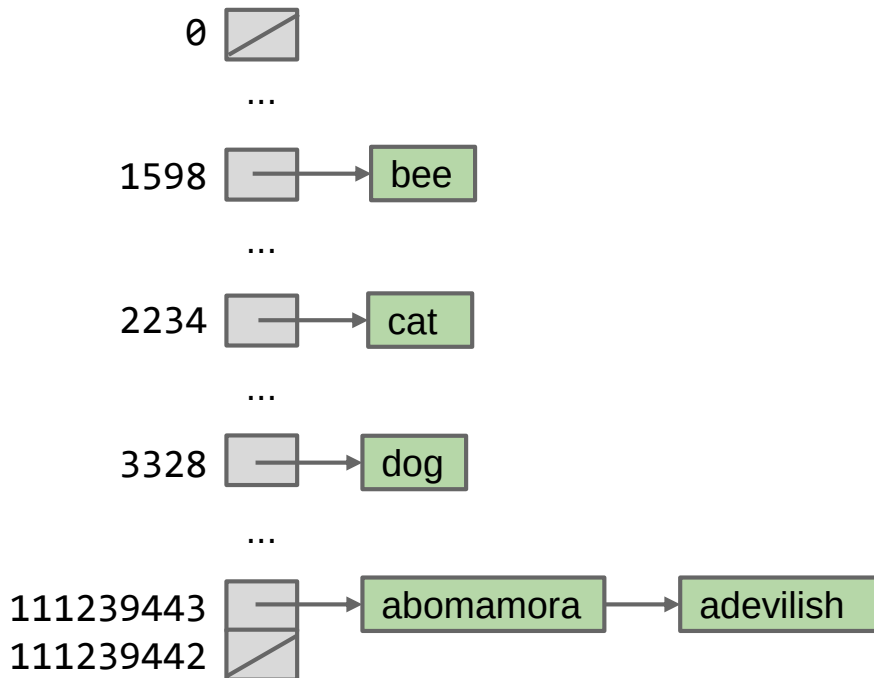
```
add("a")  
add("abomamora")  
add("adevilish")  
add("abomamora")  
contains("adevilish")
```



→ return true

Separate Chaining Performance

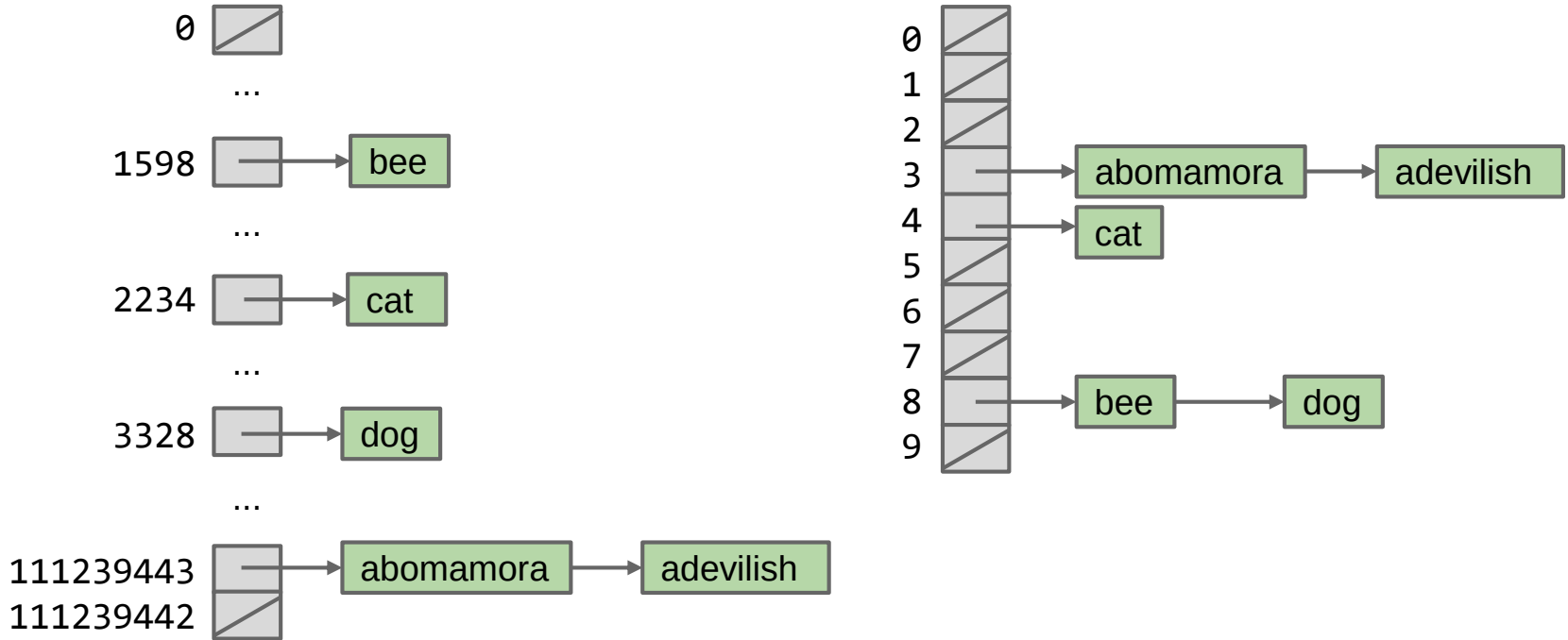
- The running time of add and contain is $O(Q)$, where Q is the length of the longest list
 - no longer $O(1)$, since to add, need to check whether the item is already in the list



let us improve our design.
notice that we don't need
billions of buckets.
what can we do to save space?

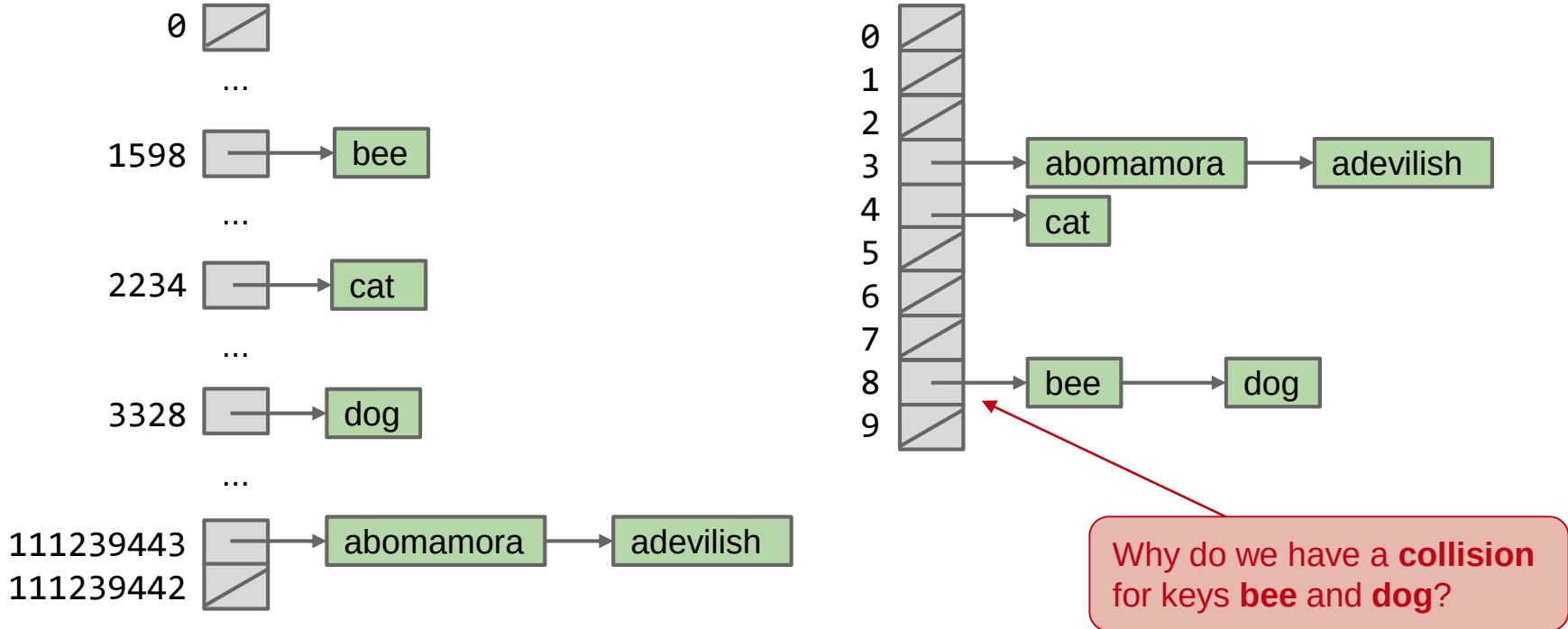
Saving Memory

- We could reduce the number of buckets, and use modulus of hash code to compute index
 - for example, use 10 buckets, and compute bucket number = hash code % 10



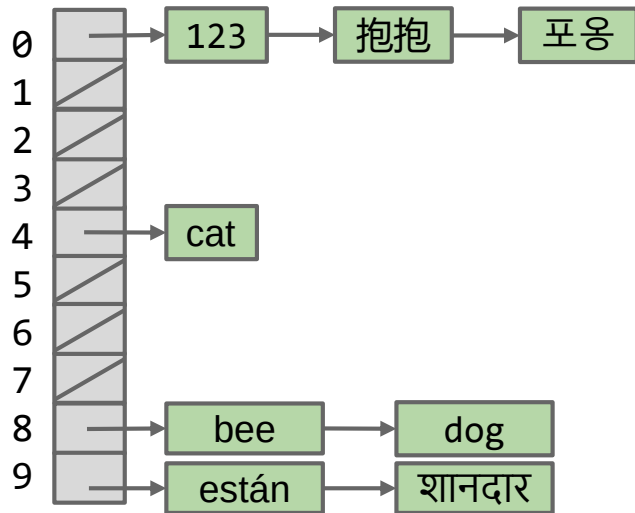
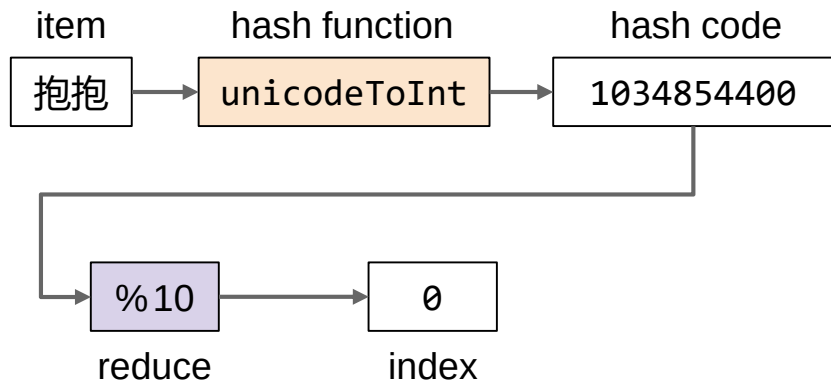
In-Class Quiz 4

- We could reduce the number of buckets, and use modulus of hash code to compute index
 - for example, use 10 buckets, and compute bucket number = hash code % 10



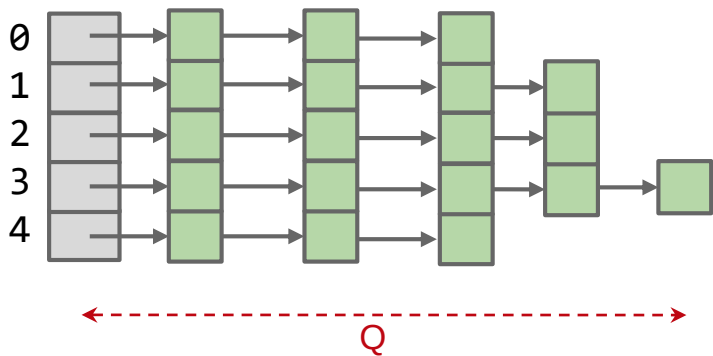
Hash Table

- We have just created the so-called **hash table** !
 - The item is converted by a **hash function** into an integer representation called a **hash code**
 - The hash code is then reduced to a **valid index**, usually using the modulus operator



Hash Table Performance

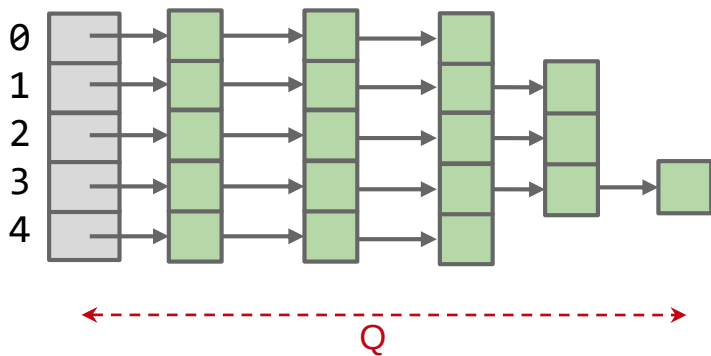
- Hash table uses way less memory/space than INTSet or ENGSet, but now the running time is no longer constant $O(1)$, it becomes $O(Q)$
 - where Q is the length of the longest list



for this hash table that has 5 buckets, and stores N items, what is the order of growth of Q in terms of N ?

Hash Table Performance

- Hash table uses way less memory/space than INTSet or ENGSet, but now the running time is no longer constant $O(1)$, it becomes $O(Q)$
 - where Q is the length of the longest list

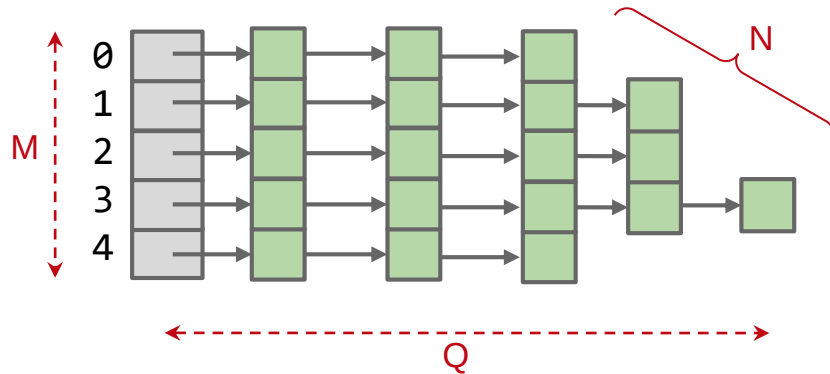


for this hash table that has 5 buckets, and stores N items, what is the order of growth of Q in terms of N ?

In the *best case*, the items are **evenly distributed**, the length of the longest list will be $N/5$.
In the *worst case*, all items in **a single list**, it will be N .
So, Q is $O(N)$, which is not good...

Improving Hash Table

- Suppose we have:
 - A fixed number of buckets M
 - An increasing number of items N



- Even if items are spread out evenly, lists are of length $Q = N/M$
 - For $M = 5$, that means $Q = O(N)$
 - Results in linear time operations, which is bad
- How can we improve our design to guarantee that N/M is $O(1)$?

Improving Hash Table

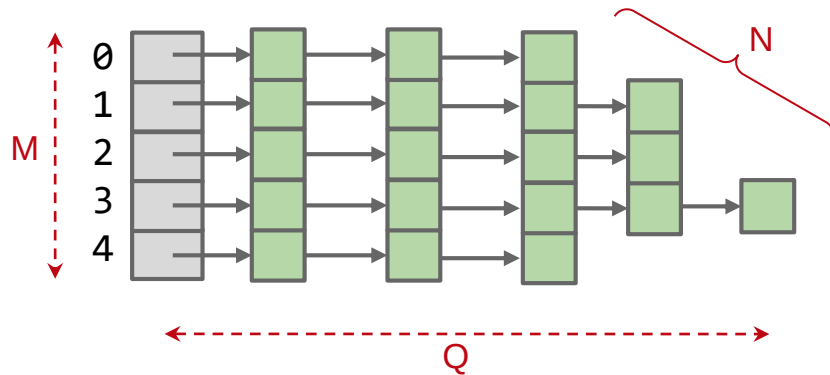
- Suppose we have:

- ~~○ A fixed~~ number of buckets M

An increasing

- An increasing number of items N

improve the ratio by growing M at the same rate of N



- Even if items are spread out evenly, lists are of length $Q = N/M$
 - For $M = 5$, that means $Q = O(N)$
 - Results in linear time operations, which is bad
- How can we improve our design to guarantee that N/M is $O(1)$?

Improving Hash Table 2

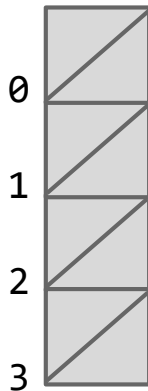
- As long as $M = O(N)$, then $O(N/M) = O(1)$
- Idea: When N/M is ≥ 1.5 , then double M
 - when it gets too full, double the buckets!
 - we call this process of increasing M : **resizing**
 - N/M is often called the **load factor**
 - it represents how full the hash table is
- This rule ensures that the average list is never more than 1.5 items long!

Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

suppose we start with 4 empty buckets

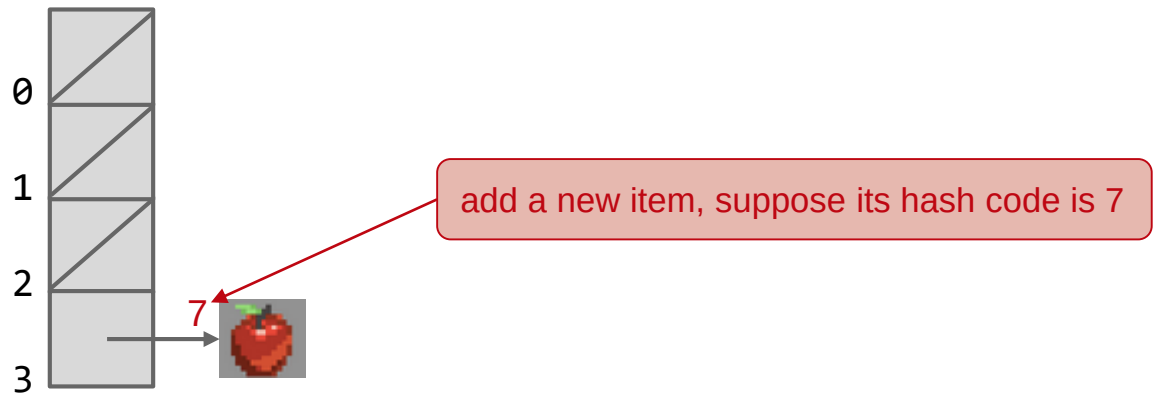
$N = 0$ $M = 4$ $N/M = 0$



Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

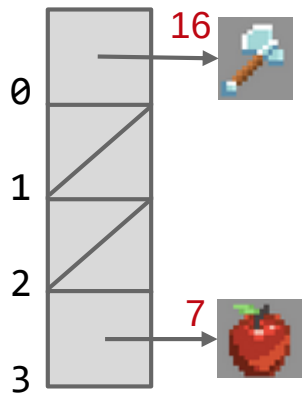
$N = 1$ $M = 4$ $N/M = 0.25$



Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

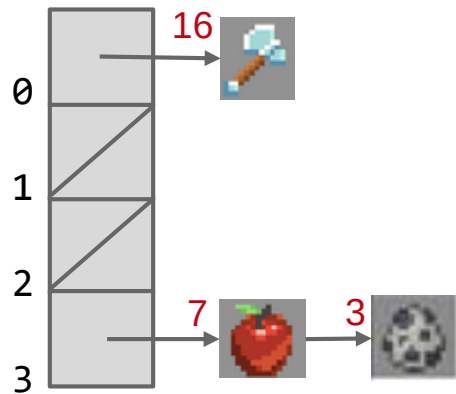
$N = 2$ $M = 4$ $N/M = 0.5$



Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

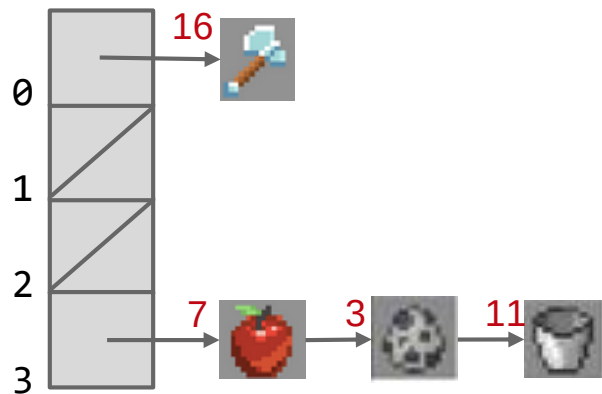
$N = 3$ $M = 4$ $N/M = 0.75$



Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

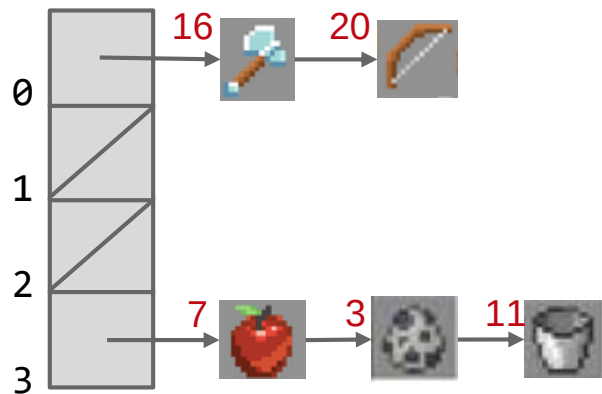
$N = 4$ $M = 4$ $N/M = 1$



Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

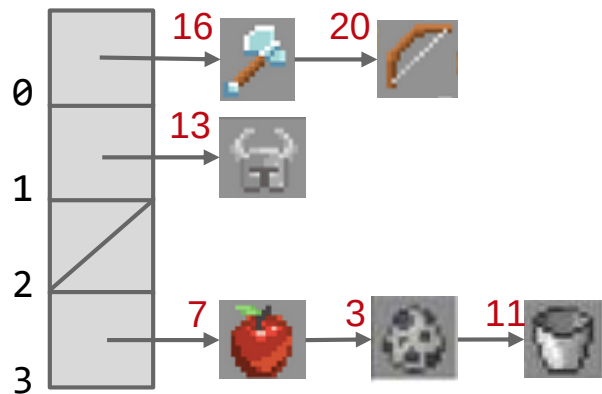
$N = 5$ $M = 4$ $N/M = 1.25$



Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

$N = 6$ $M = 4$ $N/M = 1.5$



N/M is too large,
it's time to double!

In-Class Quiz 5

- Idea: When N/M is ≥ 1.5 , then double M

where do we put this item
after doubling?

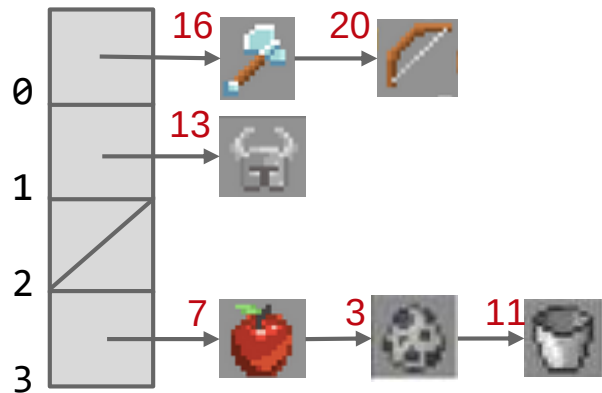


?

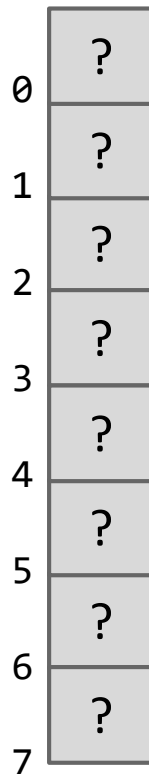
$N = 6$

$M = 4$

$N/M = 1.5$



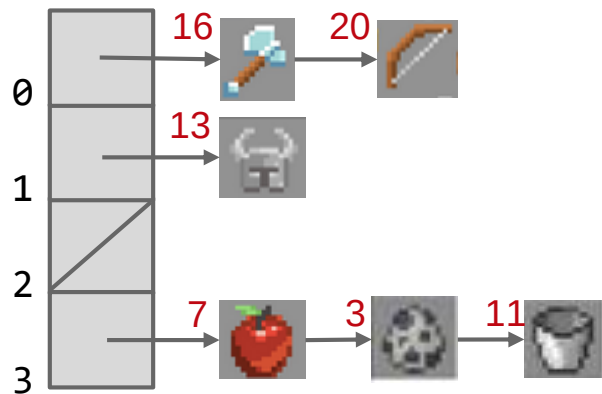
N/M is too large,
it's time to double!



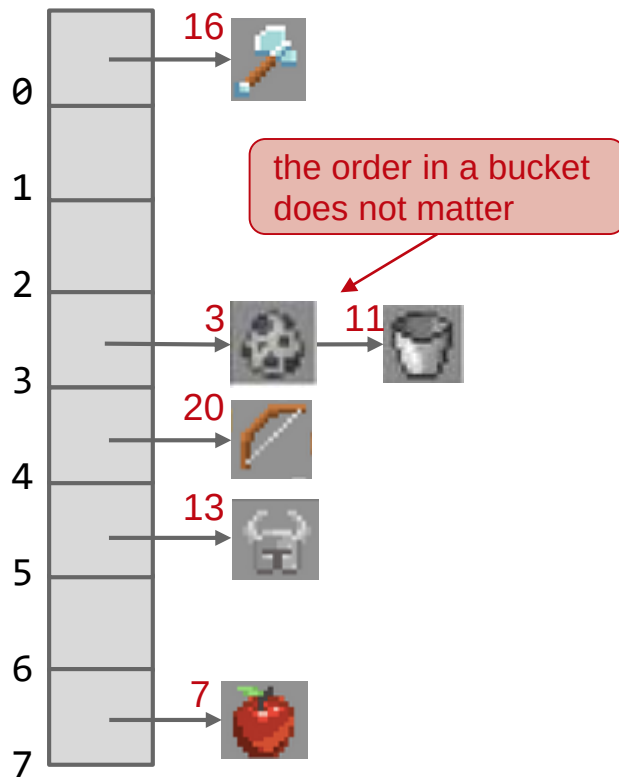
Improving Hash Table Example

- Idea: When N/M is ≥ 1.5 , then double M

$N = 6$ $M = 4$ $N/M = 1.5$



$N = 6$ $M = 8$ $N/M = 0.75$



Resizing Hash Table Running Time

- As long as $M = O(N)$, that is, grows linearly, then $O(N/M) = O(1)$
- *Assuming items are evenly distributed*, lists will be approximately N/M items long, resulting in $O(N/M)$ runtimes
 - Our doubling strategy ensures that $N/M = O(1)$
 - Thus, worst case runtime for all operations is $O(N/M) = O(1)$
 - unless that operation causes a resize
- What is the runtime of resizing?

Resizing Hash Table Running Time

- As long as $M = O(N)$, that is, grows linearly, then $O(N/M) = O(1)$
- *Assuming items are evenly distributed*, lists will be approximately N/M items long, resulting in $O(N/M)$ runtimes
 - Our doubling strategy ensures that $N/M = O(1)$
 - Thus, worst case runtime for all operations is $O(N/M) = O(1)$
 - unless that operation causes a resize
- Resizing takes $O(N)$ time, because we have to redistribute all items!
 - Most add operations will be $O(1)$, but some will be $O(N)$ time to resize
 - Similar to the array-based list, as long as we resize by a multiplicative factor, the average runtime will still be $O(1)$

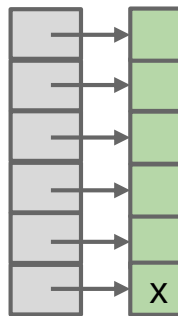
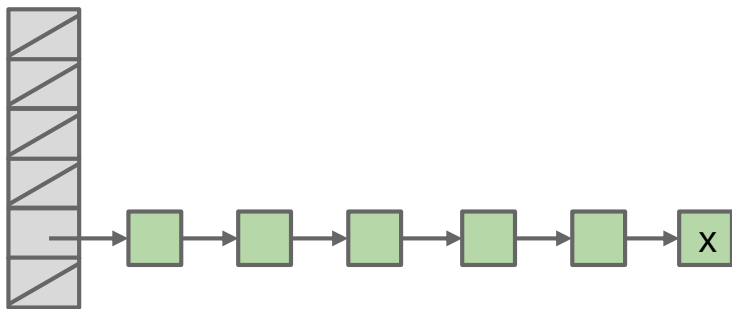
Even Distribution

- Thus, Resizing Hash Table operations are on average constant time if:
 1. we double M to ensure constant average bucket length
 2. items are evenly distributed

Even Distribution

- Thus, Resizing Hash Table operations are on average constant time if:
 1. we double M to ensure constant average bucket length
 2. items are evenly distributed
- Even distribution of item is critical for good hash table performance
 - both tables below have load factor of $N/M = 1$
 - but left table is much worse, contains is $O(N)$ for item x

we need to discuss how to ensure even distribution !



Object's Hash Code

- In Java, every class inherits the hashCode() method from Object class

Modifier and Type	Method	Description
protected Object	clone()	Creates and returns a copy of this object.
boolean	equals (Object obj)	Indicates whether some other object is "equal to" this one.
protected void	finalize()	Deprecated. The finalization mechanism is inherently problematic.
Class <?>	getClass()	Returns the runtime class of this Object.
int	hashCode()	Returns a hash code value for the object.
void	notify()	Wakes up a single thread that is waiting on this object's monitor.
void	notifyAll()	Wakes up all threads that are waiting on this object's monitor.
String	toString()	Returns a string representation of the object.
void	wait()	Causes the current thread to wait until it is awakened, typically by being <i>notified</i> or <i>interrupted</i> .
void	wait (long timeout)	Causes the current thread to wait until it is awakened, typically by being <i>notified</i> or <i>interrupted</i> , or until a certain amount of real time has elapsed.
void	wait (long timeout, int nanos)	Causes the current thread to wait until it is awakened, typically by being <i>notified</i> or <i>interrupted</i> , or until a certain amount of real time has elapsed.

default implementation of
hashCode returns the memory
address of the object

String's Hash Code

- Java's String overrides hashCode
 - for example:

```
System.out.println("a".hashCode());  
System.out.println("bee".hashCode());  
System.out.println("포옹".hashCode());  
System.out.println("kamala lifefully".hashCode());  
System.out.println("đậu hũ".hashCode());
```

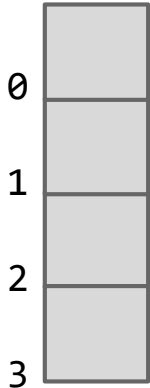
→ 97
→ 97410
→ 1732557
→ 1732557
→ -2108180664

a collision !

a negative !

Negative Hash Code in Java

- Suppose that the hash code of an item is -1
- Unfortunately, $-1 \% 4 \rightarrow -1$
 - will result in index errors!
- Solution: use `Math.floorMod` instead



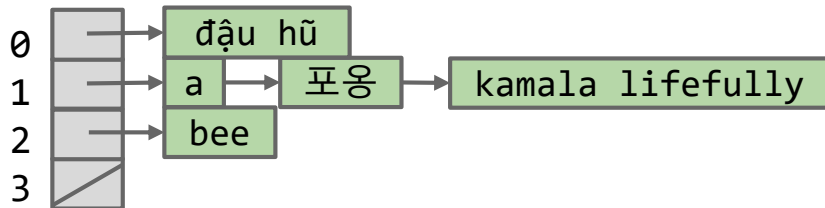
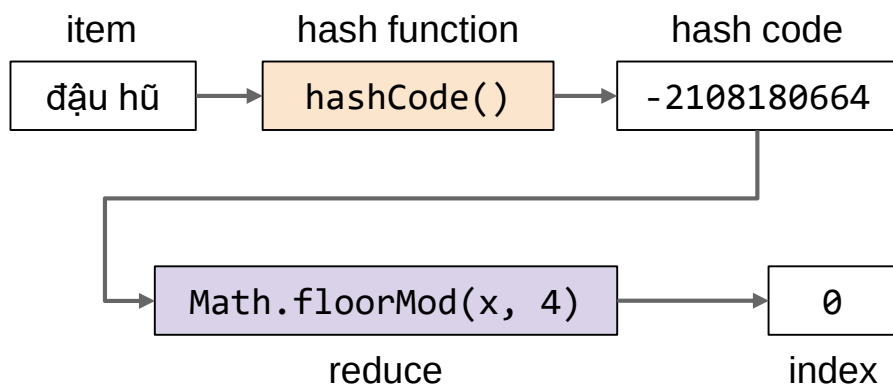
```
public class ModTest {  
    public static void main(String[] args) {  
        System.out.println(-1 % 4);  
        System.out.println(Math.floorMod(-1, 4));  
    }  
}
```

→ -1

→ 3

Our Hash Table Set Implementation HashSet

- Here is our set implementation using hash tables in Java, called HashSet:
 - item is converted by the **hashCode** method into an integer representation called a **hash code**
 - the hash code is then reduced to a **valid index**, using the `Math.floorMod` method



HASet: Member Variables and Constructor

- Here is our hash-based implementation of set

```
/*  
 * Hash-based Set  
 */  
public class HASet<T> {  
  
    private static final int DEFAULT_CAPACITY = 16;  
  
    private ArrayList<ArrayList<T>> buckets;  
    private int size;  
  
    private int reduce(T item) {  
        return Math.floorMod(item.hashCode(), DEFAULT_CAPACITY);  
    }  
  
    public HASet() {  
        buckets = new ArrayList<>();  
        for (int i = 0; i < DEFAULT_CAPACITY; i++) {  
            buckets.add(new ArrayList<>());  
        }  
        size = 0;  
    }  
}
```

as a rep of a (fixed) hash table,
we use ArrayList of ArrayList

hash function and reduce

initializing the buckets

HashSet: Contains

- Recall that in a set, an item is either in a set or not

```
/**
 * Checks whether an item is inside the set.
 * @param item to be checked
 * @return true iff the set contains the item
 */
public boolean contains(T item) {
    if (item == null) {
        return false;
    }
    int index = reduce(item);
    return buckets.get(index).contains(item);
}
```

compute the index,
go to the right bucket,
and call ArrayList.contains

HASet: Add

- Recall that in a set, we must have no duplicates

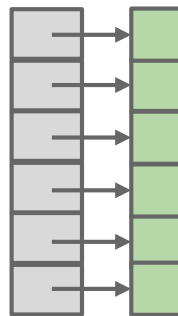
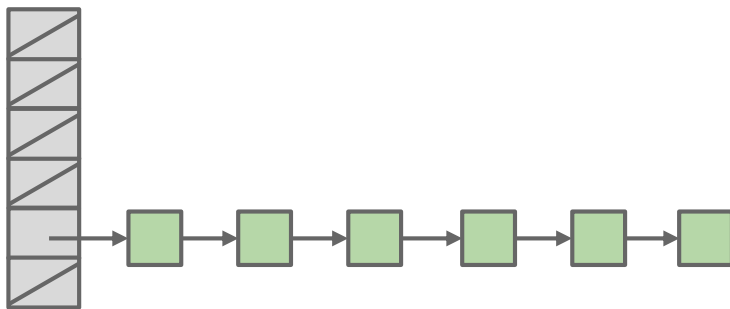
```
/**
 * Adds an item into the set if it is not already inside.
 * @param item to be added inside the set.
 * @throws IllegalArgumentException if item is null.
 */
public void add(T item) {
    if (item == null) {
        throw new IllegalArgumentException();
    }
    if (contains(item)) {
        return;
    }
    int index = reduce(item);
    buckets.get(index).add(item);
    size++;
}
```



compute the index,
go to the right bucket,
and add the item,
if it is not already in there

Bad hashCode

- The following are examples of bad hash functions
 - return 0
 - just returning the first character of a word, e.g. "cat" → 3
 - adding chars together, e.g. "ab" collides with "ba"
- Recall that we want hash tables that look like the table on the right:



String hashCode Method

- The following is the Java 8 hashCode for String:

```
@Override
public int hashCode() {
    int h = cachedHashValue;
    if (h == 0 && this.length() > 0) {
        for (int i = 0; i < this.length(); i++) {
            h = 31 * h + this.charAt(i);
        }
        cachedHashValue = h;
    }
    return h;
}
```

it cached the value, only compute if it is not computed before

it uses a small base

Choosing a Base

- Java's hashCode() function for Strings:
 - $h(s) = s_0 \times 31^{n-1} + s_1 \times 31^{n-2} + \dots + s_{n-1}$
- Our asciiToInt function for Strings:
 - $h(s) = s_0 \times 126^{n-1} + s_1 \times 126^{n-2} + \dots + s_{n-1}$
- Which one is better?
 - recall that overflow is problem for base 126
 - and in turns, that causes collision

Base 126

- **Major** collision problem:
 - "Laughter is the best thing on the earth.".hashCode() yields 634199182
 - "Muesli is the best thing on the earth.".hashCode() yields 634199182
 - "Being heard is the best thing on the earth.".hashCode() yields 634199182
 - "Memes is the best thing on the earth.".hashCode() yields 634199182
- Any string **that ends in the same last 32 characters** has the same hash code !
 - Why? Because of overflow
 - The causes $126^{32} = 126^{33} = 126^{34} = \dots = 0$
 - Thus, upper characters are all multiplied by zero

Typical Base

- A typical hash code base is a **small prime**
- Why prime?
 - **Never even:** avoid the overflow issue on previous slide
 - **Lower chance** of resulting hash code having a bad relationship with the number of buckets, e.g. the hash code is a multiple of the number of buckets
- Why small?
 - **Lower cost** to compute

Collection hashCode Method

- Suppose we have a collection of items, and each item has its own hashCode
 - somewhat similar to the one in String:

```
@Override
public int hashCode() {
    int hashCode = 1;
    for (Object o : this) {
        hashCode = hashCode * 31;
        hashCode = hashCode + o.hashCode();
    }
    return hashCode;
}
```

iterate over all items

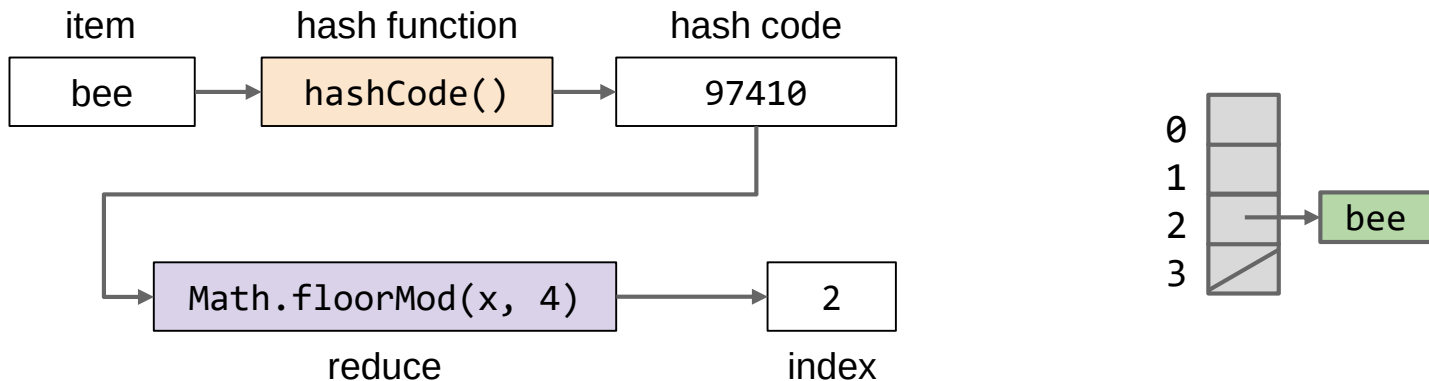
"raise" to the "next power"
with small prime

add the hashCode of an item

we may only look at the first few items,
since the Collection maybe very large

Summary for Hash Table

- Hash table:
 - Item is converted into a hash code using good hashCode method
 - The hash code is then reduced to a valid index
 - Item is then stored in a bucket corresponding to that index
- Resizing hash table:
 - Double when load factor N/M exceeds some constant, say 1.5 or 0.75
 - If items are spread out nicely, you get $O(1)$ average runtime



Thank you for your attention !

- **In this lecture, you have learned:**
 - to create a hash table data structure for fast adding and searching
 - to handle collisions with separate chaining
 - to create a resizing hash table that enables you to add and search data in constant average time
 - to create good hash codes

- Please continue to **Exercise 13**